



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

12. Entwurfsprinzipien



Entwurf von Algorithmen

- Aufgabenbeschreibung
 - meist informelle Spezifikation
 - selten alle Sonderfälle beschrieben
- Daraus Algorithmus entwickeln?
 - nicht automatisierbar!
- Konstruktive & kreative Tätigkeit!
 - Hilfe durch
 - Entwurfsprinzipien
 - Entwurfsmuster,
 - Best practices



Entwurfsprinzipien

- Schrittweise Verfeinerung
- Rekursion
- Einsatz von Algorithmenmustern



Entwurfsprinzip: Schrittweise Verfeinerung

- Schrittweise Verfeinerung von Pseudo-Code-Algorithmen
- Ersetzen von Pseudo-Code-Teilen durch
 - verfeinerten Pseudo-Code
 - Programmiersprachen-Code
- Beispiel
 - Gib Kaffeepulver in Tasse
 - ↳
 - Öffne Kaffeeglas
 - Entnehme Kaffee mit Löffel
 - Kippe Löffel in Tasse
 - Schließe Kaffeeglas



Schrittweise Verfeinerung und Codierung

- Fakultät als Prozeduraufruf

Factorial (n)

- Fakultät als Algorithmus

```
Factorial (X);  
  Y := 1;  
  while X > 1 do  
    Y := Y*X; X := X-1  
  od  
  output Y
```

- Fakultät als Code



```
public static int factorial(int n) {  
  // ...  
  return 0;  
}
```



Entwurfsprinzipien – Rekursion

- Rekursion nicht im Ingenieurwesen entwickelt, sondern typisch für Informatik (und Mathematik)
- Größere Probleme (Größe der Eingabe) auf kleinere Instanzen desselben Problems reduzieren.
- Einfache Fälle direkt lösen
- Lösung ohne Rekursion auch immer möglich
 - Imperative Algorithmen sind berechnungsuniversell
 - Umwandlung **iterativ** $\Leftrightarrow$ **rekursiv** ist automatisierbar



Entwurfsprinzipien - Algorithmenmuster

- Anpassung von generischen Algorithmenmustern für bestimmte Problemklassen an eine konkrete Aufgabe
 - Dokumentation des Lösungsverfahrens am Beispiel eines einfachen Vertreters der Problemklasse
 - Bibliothek von Strategien („design pattern“, „best practice“) zur Ableitung eines abstrakten Programmrahmens
 - Programmiersprachenunterstützung durch parametrisierte Algorithmen und Vererbung



Algorithmenmuster Greedy

- Greedy = „gierig“
- Prinzip: Versuche mit jedem Teilschritt so viel wie möglich zu erreichen.
- Greedy-Algorithmen
 - Greedy-Algorithmen (gierige Algorithmen) zeichnen sich dadurch aus, dass sie immer denjenigen Folgezustand auswählen, der zum Zeitpunkt der Wahl den größten Gewinn bzw. das beste Ergebnis verspricht

Beispiel Algorithmenmuster Greedy

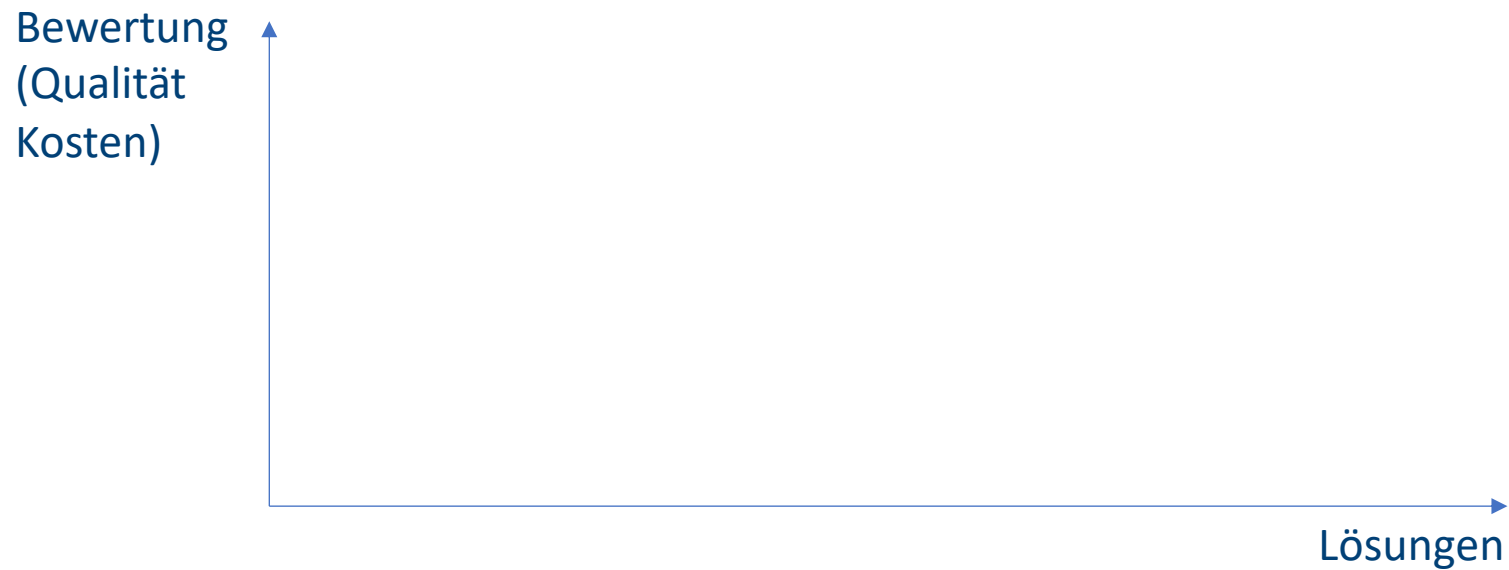
- Herausgabe von Wechselgeld auf Beträge unter 1 Euro
- verfügbare Münzen mit Werten zu 50, 20, 10, 5, 2, 1 Cent
- Ziel: so wenig Münzen wie möglich
- Beispiel:
 $78 \text{ Cent} = 50 + 20 + 5 + 2 + 1$
- Algorithmus:
 1. Nehme jeweils immer die größte Münze unter Zielwert, und ziehe sie von diesem ab.
 2. Verfahre derart bis Zielwert gleich Null.





Problem: lokales Optimum

- Greedy-Algorithmen berechnen in jedem Schritt ein lokales Optimum, das globale Optimum kann verfehlt werden





Beispiel Algorithmenmuster Greedy

- Greedy-Algorithmen berechnen in jedem Schritt **lokales Optimum** $\rightsquigarrow$ globales Optimum kann verfehlt werden!
- Beispiel:
 - Münzen: 25, 10, und 1
 - Zielwert 30
 - Greedy: $25 + 1 + 1 + 1 + 1 + 1$
 - Optimum: $10 + 10 + 10$
- aber in vielen Fällen entsprechen lokale Optima den globalen oder es reicht ein lokales Optimum aus!





Problemklassen für Greedy-Algorithmen

- Gegeben ist Menge von Eingabewerten
- Menge von Lösungen, die aus Eingabewerten aufgebaut sind
- Lösungen lassen sich schrittweise aus partiellen Lösungen, beginnend bei der leeren Lösung, durch Hinzunahme von Eingabewerten aufbauen
- Bewertungsfunktion für partielle und vollständige Lösungen
- Gesucht wird die / eine optimale Lösung



Optimaler Spannbaum in gewichtetem Graphen

- Gegeben ein gewichteter Graph
 - $G = (V, E)$
 - mit $w: E \rightarrow \mathbb{R}$ Gewicht/Länge
(Annahme: alle Gewichte verschieden)
- Gesucht: der Spannbaum B mit der kleinsten Summe von Gewichten aller Kanten.
- Greedy
 - eine Menge von Eingabewerten: Knoten + Kanten
 - Menge von Lösungen: alle Spannbäume
 - Spannbaum eines Teilgraphen erweitern?
 - Bewertungsfunktion: Gesamtgewicht
 - Optimum (Minimum) ist gesucht



Optimaler Spannbaum: Greedy

- Gegeben eine Teilmenge der Knoten / Kanten
 - und ein optimaler Spannbaum für diese Knoten
- Füge einen Knoten / Kante dazu
- und erweitere den Spannbaum, so dass er weiterhin optimal ist.
- Zwei Möglichkeiten:
 - Teilmenge ist und bleibt zusammenhängend (Prim)
 - Teilmenge kann unzusammenhängend sein (Wald)
 - charakterisiert durch Menge von Kanten (Kruskal)



Formalisierung

```
algorithm Kruskal(V, E, w)
  Eingabe V: Knotenmenge, E: Kantenmenge, w: E → ℝ Gewicht
  E1 := ∅
  while (V, E1) nicht zusammenhängend do
    wähle die kleinste Kante e ∈ E \ E1,
      die zwei Komponenten in (V, E1) verbindet
    {oder: die keinen Kreis erzeugt}
    E1 := E1 ∪ {e}
  od
```




Algorithmenmuster Teile-und-Herrsche

- Divide et impera
- Divide and conquer



Algorithmenmuster Teile-und-Herrsche

- Grundidee:
 - Teile das gegebene Problem in mehrere (oft zwei) getrennte **Teilprobleme** auf,
 - löse diese einzeln
 - und setze die Lösungen des ursprünglichen Problems aus den Teillösungen zusammen
- Wende dieselbe Technik auf jedes der Teilprobleme an, dann auf deren Teilprobleme usw., bis die Teilprobleme klein genug sind, dass man eine Lösung explizit angeben kann.
- Strebe an, dass jedes Teilproblem von derselben Art ist wie das ursprüngliche Problem, so dass es mit demselben Algorithmus gelöst werden kann.



Algorithmenmuster Teile-und-Herrsche

```
procedure DIVANDCONQ (P: problem )
begin
  :
  if [P klein ]
  then [ explizite Lösung ]
  else [ teile P auf in P1 , ... , Pk ]
      DIVANDCONQ( P1 ) ;
      :
      DIVANDCONQ( Pk ) ;
      [ setze Lösung für P aus Lösungen
        für P1 , ... , Pk zusammen ]
  fi
end
```



Algorithmenmuster Teile-und-Herrsche

- Beispiele
 - Binäre Suche
 - MergeSort
 - QuickSort

```
if F einelementig then
    return F
else
    Teile F in F1 und F2;
    F1 := MergeSort( F1 );
    F2 := MergeSort( F2 );
    return Merge( F1, F2 )
fi
```



Algorithmenmuster Backtracking

- Auch exhaustive search, erschöpfende Suche



Algorithmenmuster Backtracking

- Backtracking: Versuch-und-Irrtum-Prinzip (trial and error)
 - versuche, erreichte Teillösung schrittweise zu einer Gesamtlösung auszubauen
 - falls Teillösung nicht zu einer Lösung führen kann → letzten Schritt bzw. die letzten Schritte zurücknehmen und stattdessen alternative Wege probieren
 - alle in Frage kommenden Lösungswege werden ausprobiert
 - vorhandene Lösung wird entweder gefunden (unter Umständen nach sehr langer Laufzeit) oder es existiert definitiv keine Lösung



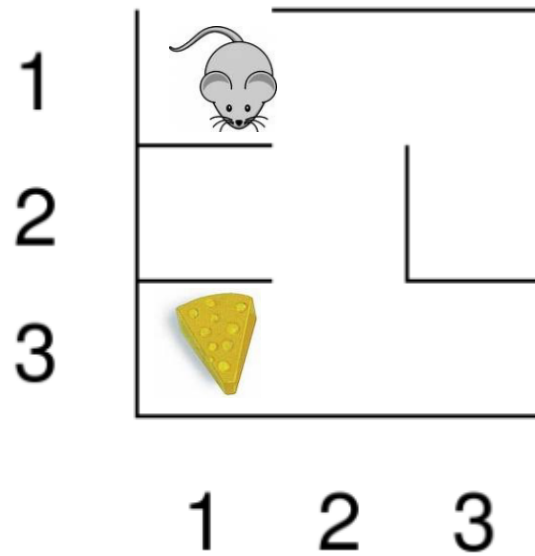
Algorithmenmuster Backtracking

- Backtracking („zurückverfolgen“): allgemeine systematische Suchtechnik
 - KF Menge von Konfigurationen K
 - K_0 Anfangskonfiguration
 - für jede Konfiguration K_i direkte Erweiterungen: $K_{i,1}, \dots, K_{i,n_i}$
 - für jede Konfiguration ist entscheidbar, ob sie eine Lösung ist
- Aufruf: **BACKTRACK**(K_0)
 - Terminierung? Nur wenn
 - Lösungsraum endlich ist und
 - Kreisen im Lösungsraum ausgeschlossen ist.



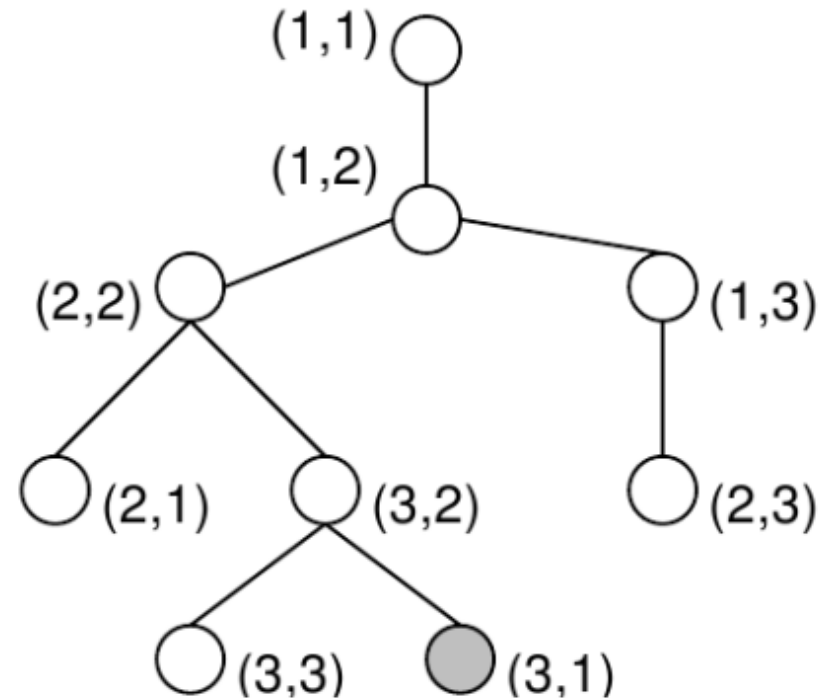
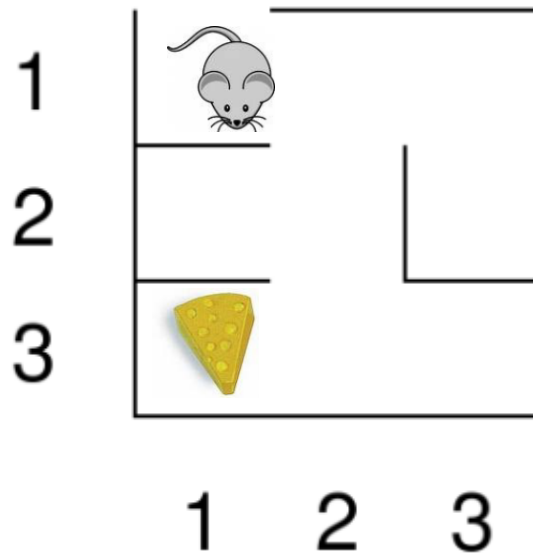
Labyrinth-Suche

- Wie findet die Maus den Käse?



Labyrinth-Suche

- Wie findet die Maus den Käse?





Algorithmenmuster Backtracking

```
procedure BACKTRACK (K : Konfiguration )  
begin  
    ...  
    if [ K ist Lösung ] then  
        [ gib K aus ]  
    else  
        foreach [jede direkte Erweiterung K' von K] do  
            BACKTRACK( K' )  
        od  
    fi  
end
```



Algorithmenmuster Backtracking

- Typische Einsatzfelder des Backtracking
 - Spielprogramme (Schach, Dame, Labyrinthsuche, ...)
 - Erfüllbarkeit von logischen Aussagen (logische Programmiersprachen)
 - Planungsprobleme, Konfigurationen
 - logistische Fragestellungen
 - kürzeste Wege, optimale Verteilungen, ...
 - Färben von Landkarten



Algorithmenmuster Backtracking

- Varianten Backtracking
 - Bewertete Lösungen: Am Ende beste Lösung auswählen
 - Abbruch nach erster Lösung
 - Branch-and-bound: Nur Zweige verfolgen, die eine Lösung prinzipiell zulassen
 - oder: Zweige abbrechen, wenn bisher gefundenes Maximum nicht mehr übertroffen werden kann.
 - Maximale Rekursionstiefe
 - Spiele: Stellung, z.B. beim Schach heuristisch bewerten



Mehr als ein Muster: Dynamische Programmierung

- Kombination der vorherigen Muster
 - Greedy: Wahl einer optimalen Teillösung
 - Teile- und Herrsche bzw. Backtracking: rekursive Herangehensweise
- Zunächst kleine Teilprobleme lösen und speichern
- Mit den gespeicherten Werten größere Teilprobleme lösen bis Gesamtproblem gelöst ist
- Prinzip: Rechenzeit gegen Speicherplatz eintauschen
 - Zwischenergebnisse speichern und wiederverwenden
 - Vermeiden von wiederholten Auswertungen
 - Möglicherweise hoher Speicherbedarf



Mehr als ein Muster: Dynamische Programmierung

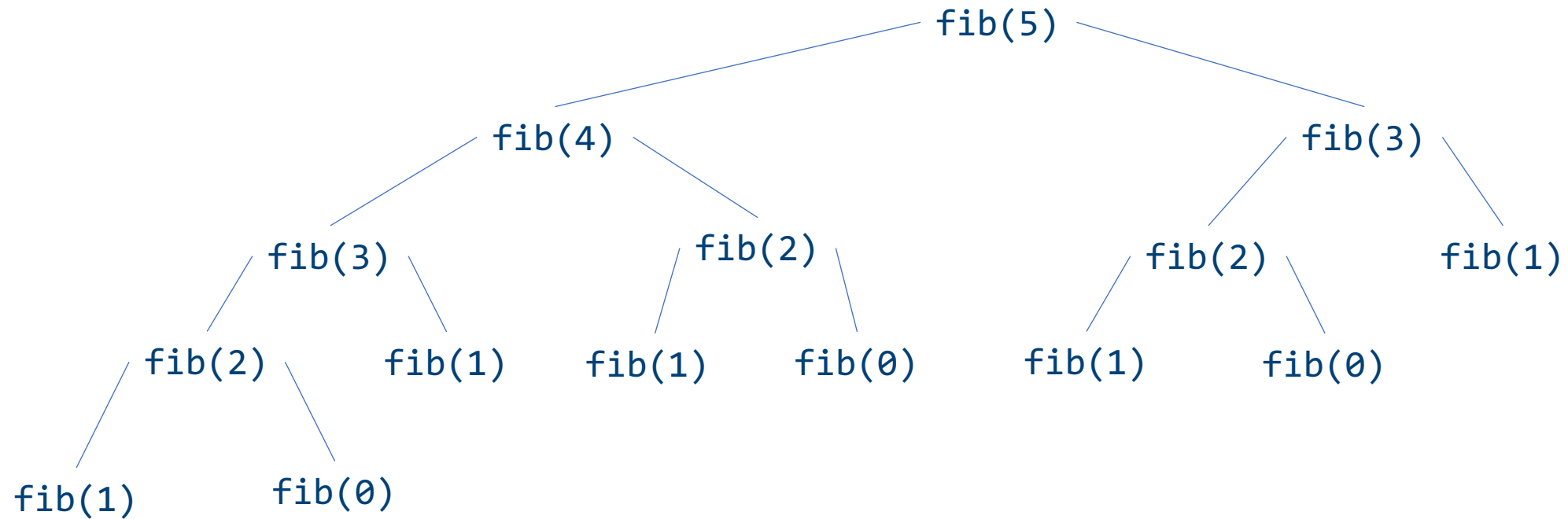
- Beispiel: Fibonacci-Zahlen
 - $f(0) = 0$
 - $f(1) = 1$
 - $f(n) = f(n - 1) + f(n - 2)$, falls $n \geq 2$

```
procedure fib (n):  
    if (n == 0) or (n == 1)  
    then return n  
    else return fib(n - 1) + fib(n - 2)
```



Beispiel: Fibonacci-Zahlen

Aufrufbaum:





Mehr als ein Muster: Dynamische Programmierung

- Vorgehen

1. Rekursive Beschreibung des Problems P
2. Bestimmung einer Menge T , die alle Teilprobleme von P enthält, auf die bei der Lösung von P – auch in tieferen Rekursionsstufen – zurückgegriffen wird.
3. Bestimmung einer Reihenfolge $T_0, \dots, T_k$ der Probleme in T , so dass bei der Lösung von T_i nur auf Probleme T_j mit $j < i$ zurückgegriffen wird.
4. Sukzessive Berechnung und Speicherung von Lösungen für $T_0, \dots, T_k$.



Beispiel: Fibonacci-Zahlen

1. Rekursive Definition der Fibonacci-Zahlen nach gegebener Gleichung.
2. $T = \{ f(0), \dots, f(n-1) \}$
3. $T_i = f(i), i = 0, \dots, n - 1$
4. Berechnung von $f(i)$ benötigt von den früheren Problemen nur die zwei letzten Teillösungen $f(i-1)$ und $f(i-2)$ für $i \geq 2$.



Memoisierte Version Fibonacci-Zahlen

```
procedure fib (n):  
    F[0] := 0; F[1] := 1;  
    for i := 2 to n do  
        F[i] := F[i-1] + F[i-2];  
    od  
    return F[n]
```



Version Fibonacci-Zahlen: konstanter Platzbedarf

```
procedure fib (n):  
    v = 0;  
    l = 1;  
    fib = 0;  
    for i := 2 to n do  
        fib = v + l;  
        v = l; l = fib;  
    od  
    if n <= 1 then return n else return fib fi
```



Zusammenfassung

- Algorithmenmuster als „Best-Practice“-Lösungen für neue Probleme
- Konkret:
 - Greedy,
 - Divide-and-Conquer,
 - Backtracking,
 - Dynamische Programmierung